boba_extension_dim06

Dimension 06: Magnet Link Detection & Parsing Algorithms

Comprehensive Research Report

Date: 2026-01-19 **Scope**: Magnet URI specification, detection regex, parsing algorithms, validation, edge cases, and implementation strategies for browser extensions

---

Table of Contents

---

1. Magnet URI Specification Reference

1.1 Official Specification (BEP 9)

Claim: The magnet URI format is defined in BEP 9 (BitTorrent Enhancement Proposal 9), which specifies the exact format and all parameters. Source: BitTorrent.org Official BEP 9 URL: https://www.bittorrent.org/beps/bep_0009.html Date: 2017-03-26 (last modified) Excerpt:

```
v1: magnet:?xt=urn:btih:<info-hash>&dn=<name>&tr=<tracker-url>&x.pe=<peer-address>
v2: magnet:?xt=urn:btmh:<tagged-info-hash>&dn=<name>&tr=<tracker-url>&x.pe=<peer-address>
```

Context: BEP 9 defines the metadata exchange protocol and the magnet URI format used to join swarms without downloading a .torrent file first. Confidence: high

1.2 Format Overview

Claim: The magnet URI consists of the scheme `magnet:` followed by query parameters. The `xt` parameter is the only mandatory parameter. Source: BitTorrent.org BEP 9 URL: https://www.bittorrent.org/beps/bep_0009.html Date: 2017-03-26 Excerpt:

```
<info-hash> Is the info-hash hex encoded, for a total of 40 characters. For compatability with existing links in the wild, clients should also support the 32 character base32 encoded info-hash.
<tagged-info-hash> Is the multihash formatted, hex encoded full infohash for torrents in the new metadata format. 'btmh' and 'btih' exact topics may exist in the same magnet if they describe the same hybrid torrent.
<peer-address> A peer address expressed as hostname:port, ipv4-literal:port or [ipv6-literal]:port.
xt is the only mandatory parameter. dn is the display name that may be used by the client to display while waiting for metadata. tr is a tracker url, if there is one. If there are multiple trackers, multiple tr entries may be included. The same applies for x.pe entries. dn, tr and x.pe are all optional.
```

Context: Defines all parameters and their roles in the magnet URI Confidence: high

1.3 Complete Parameter Reference

Parameter	Name	Description	Required
`xt`	eXact Topic	URN containing the file hash (e.g., `urn:btih:...`)	**Yes**
`dn`	Display Name	Human-readable filename for display purposes	No
`tr`	TRacker	Tracker URL (URL-encoded); multiple `tr` params allowed	No
`xl`	eXact Length	File size in bytes	No
`xs`	eXact Source	Direct download source URL or P2P source	No
`as`	Acceptable Source	Fallback web server source	No
`ws`	Web Seed	Payload data served over HTTP(S) (BEP 19)	No
`kt`	Keyword Topic	Search terms instead of a specific file hash	No
`mt`	Manifest Topic	URI pointing to a manifest/list of magnets	No
`so`	Select Only	File indices to download (BEP 53)	No
`x.pe`	PEer	Fixed peer addresses (hostname:port, ipv4:port, [ipv6]:port)	No
`x.*`	eXtension	Application-defined experimental parameters	No

Claim: Multiple parameters of the same type can be used by appending `.1`, `.2`, etc., or by simply repeating the parameter name. Source: Wikipedia - Magnet URI scheme URL: https://en.wikipedia.org/wiki/Magnet_URI_scheme Date: 2005-01-24 Excerpt:

```
xt also allows for a group setting. Multiple files can be included by adding a count number preceded by a dot (".") to each link parameter.
 magnet:?xt.1=[ URN of the first file]&xt.2=[ URN of the second file]
```

Context: Describes how multiple files/torrents can be referenced in a single magnet link Confidence: high

---

2. BTIH (BitTorrent Info Hash) Formats

2.1 v1 Info Hash (SHA-1)

Claim: The v1 info hash is a 20-byte SHA-1 hash of the bencoded “info” dictionary of a torrent file, typically represented as 40 hexadecimal characters. Source: BitTorrent.org BEP 9 URL: https://www.bittorrent.org/beps/bep_0009.html Date: 2017-03-26 Excerpt:

```
<info-hash> Is the info-hash hex encoded, for a total of 40 characters. For compatability with existing links in the wild, clients should also support the 32 character base32 encoded info-hash.
```

Context: Defines the two acceptable encodings for v1 info hashes Confidence: high

2.2 v2 Info Hash (SHA-256)

Claim: BitTorrent v2 uses SHA-256 instead of SHA-1. The v2 info-hash uses the `urn:btmh:` prefix and includes a multihash format with a 2-byte prefix of `0x12 0x20` (hex: `1220`). Source: libtorrent blog - BitTorrent v2 URL: https://blog.libtorrent.org/2020/09/bittorrent-v2/ Date: 2020-09-07 Excerpt:

```
The magnet link protocol has been extended to support v2 torrents. Like the urn:btih: prefix for v1 SHA-1 info-hashes, there's a new prefix, urn:btmh: for full v2 SHA0256 info hashes. For example, a magnet link thus looks like this:
   magnet:?xt=urn:btmh:<tagged-info-hash>&dn=<name>&tr=<tracker-url>
The info-hash with the btmh prefix is the v2 info-hash in multi-
```

```
hash format encoded in hexadecimal. In practice, this means it
will have a two byte prefix of 0x12 0x20.
```

Context: Official explanation of the BitTorrent v2 magnet link format from the libtorrent author Confidence: high

2.3 Hash Format Summary

Format	Prefix	Encoding	Length	Hash Function	Example
v1 hex	`urn:btih:`	Base16 (hex)	40 chars	SHA-1 (20 bytes)	`d2474e86c95b19b8bcfdb92b`
v1 base32	`urn:btih:`	Base32 (RFC 4648)	32 chars	SHA-1 (20 bytes)	`QHQXPYWMACKDWKP47RRVIV7V`
v2 multihash	`urn:btmh:`	Hex with `1220` prefix	64 chars after prefix	SHA-256 (32 bytes)	`122067EFB3C2FA20CC493979`
Hybrid	Both prefixes	Both encodings	40 + 64 chars	Both SHA-1 and SHA-256	Contains both `urn:btih:`

Claim: The `1220` prefix in v2 info hashes is the multihash format. `0x12` = SHA-256, `0x20` = 32 bytes. Source: GitHub Issue - simple-torrent URL: https://github.com/boypt/simple-torrent/issues/146 Date: 2021-10-25 Excerpt:

```
The info-hash with the btmh prefix is the v2 info-hash in multi-
hash format encoded in hexadecimal. In practice, this means it
will have a two byte prefix of 0x12 0x20. It is possible to
include both a v1 (btih) and v2 (btmh) info-hash in a magnet link,
for backwards compatibility.
For example the following is a valid v2 magnet link:
 magnet:?
xt=urn:btmh:122067EFB3C2FA20CC493979084FB44C6A93F79442AFA4A2E01C898
```

Context: Shows a concrete example of a valid v2 magnet link with the 1220 prefix Confidence: high

---

3. Comprehensive Regex Patterns for Detection

3.1 Battle-Tested Detection Regex

Claim: A comprehensive magnet link detection regex must account for `magnet:?` prefix, `xt` parameter with various URN types, and all valid infohash formats. Source: Stack Overflow + magnet-uri source code analysis URL: https://stackoverflow.com/questions/8227280/any-way-to-verifiy-a-magnet-link-javascript Date: 2011-11-22 (with updates) Excerpt:

```
// Basic validation (Stack Overflow community)
/magnet:\?xt=urn:[a-z0-9]+:[a-z0-9]{32}/i

// Improved to handle both base32 (32) and hex (40) and btmh (64
        with prefix)
```

Context: Early regex attempts, evolved to handle more formats Confidence: medium

3.2 Production-Grade Detection Regex

```
/**
 * Comprehensive magnet link detection regex
 * Matches:
 *   - magnet:?xt=urn:btih:<40-char-hex>
 *   - magnet:?xt=urn:btih:<32-char-base32>
 *   - magnet:?xt=urn:btmh:1220<64-char-hex>
 *   - With optional dn, tr, xl, xs, as, ws, kt, mt, so, x.pe
       parameters
 *   - Multiple repeated parameters (tr=...&tr=...)
 *   - xt.1, xt.2 style numbered parameters
 */

// Core magnet link detection - matches the start and xt parameter
const MAGNET_REGEX_CORE = /magnet:\?[^\s"<>]*xt=urn:bti[h]?:[a-fA-
        F0-9]{40}(?![a-fA-F0-9])/i;

// Base32 variant
const MAGNET_REGEX_BASE32 = /magnet:\?[^\s"<>]*xt=urn:bti[h]?:[A-
        Z2-7]{32}(?![A-Z2-7])/i;

// v2 multihash variant
const MAGNET_REGEX_V2 = /magnet:\?[^\s"<>]*xt=urn:btmh:1220[a-fA-
        F0-9]{64}(?![a-fA-F0-9])/i;
```

```
// Combined comprehensive regex
const MAGNET_REGEX_FULL = /magnet:\?(?:[^\s"<>]*&)*xt=urn:bti[h]?:
        [a-fA-F0-9]{40}(?:[^\s"<>]*|$)|magnet:\?(?:
        [^\s"<>]*&)*xt=urn:bti[h]?:[A-Z2-7]{32}(?:[^\s"<>]*|$)|
        magnet:\?(?:[^\s"<>]*&)*xt=urn:btmh:1220[a-fA-F0-9]{64}(?:
        [^\s"<>]*|$)/i;
```

3.3 Recommended Detection Strategy

For browser extension content scripts scanning pages, the recommended approach is a two-phase detection:

```
/**
 * Phase 1: Fast candidate detection using a broad regex
 * Phase 2: Validation and parsing of candidates
 */

// Phase 1: Fast scan - detects potential magnet links
// This is intentionally broad to catch edge cases
const MAGNET_CANDIDATE_REGEX = /magnet:\?\S+/gi;

// Phase 2: Validate candidates have required xt parameter
const XT_PARAMETER_REGEX = /[?&]xt=([^&]+)/;

/**
 * Scan text for magnet links
 * @param {string} text - Text to scan
 * @returns {string[]} - Array of valid magnet URI strings
 */
function scanForMagnetLinks(text) {
  const candidates = text.match(MAGNET_CANDIDATE_REGEX) || [];
  const valid = [];

  for (const candidate of candidates) {
    const xtMatch = candidate.match(XT_PARAMETER_REGEX);
    if (xtMatch && isValidXt(xtMatch[1])) {
      valid.push(candidate);
    }
  }

  return valid;
}

/**
 * Validate the xt parameter value
 * @param {string} xt - The xt parameter value (e.g.,
        "urn:btih:abc123...")
 * @returns {boolean}
 */
function isValidXt(xt) {
  // v1 hex: urn:btih: + 40 hex chars
  if (/^urn:bti[h]?:[a-fA-F0-9]{40}$/i.test(xt)) return true;
```

```
  // v1 base32: urn:btih: + 32 base32 chars
  if (/^urn:bti[h]?:[A-Z2-7]{32}$/i.test(xt)) return true;
  // v2 multihash: urn:btmh:1220 + 64 hex chars
  if (/^urn:btmh:1220[a-fA-F0-9]{64}$/i.test(xt)) return true;
  return false;
}
```

3.4 Regex Test Cases

```
// Test cases for magnet link detection
const TEST_CASES = [
  // Valid v1 hex
  { input: 'magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36',
        expected: true, desc: 'v1 hex - minimal' },
  { input: 'magnet:?
        xt=urn:btih:D2474E86C95B19B8BCFDB92BC12C9D44667CFA36',
        expected: true, desc: 'v1 hex - uppercase' },
  { input: 'magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&dn=test',
        expected: true, desc: 'v1 hex with dn' },
  { input: 'magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&tr=udp%3A%2F%2Ftracke
        expected: true, desc: 'v1 hex with tracker' },

  // Valid v1 base32
  { input: 'magnet:?
        xt=urn:btih:QHQXPYWMACKDWKP47RRVIV7VOURXFE5Q', expected:
        true, desc: 'v1 base32' },
  { input: 'magnet:?
        xt=urn:btih:WRN7ZT6NKMA6SSXYKAFRUGDDIFJUNKI2', expected:
        true, desc: 'v1 base32 (qBittorrent example)' },

  // Valid v2
  { input: 'magnet:?
        xt=urn:btmh:122067EFB3C2FA20CC493979084FB44C6A93F79442AFA4A2E01C8988C1C197
        expected: true, desc: 'v2 multihash' },

  // Hybrid
  { input: 'magnet:?
        xt=urn:btih:631a31dd0a46257d5078c0dee4e66e26f73e42ac&xt=urn:btmh:1220d8dd3
        test', expected: true, desc: 'hybrid v1+v2' },

  // Invalid
  { input: 'magnet:?xt=urn:btih:SHORT', expected: false, desc:
        'too short' },
  { input: 'magnet:?dn=test', expected: false, desc: 'missing
        xt' },
  { input: 'magnet:', expected: false, desc: 'empty magnet' },
  { input: 'magnet:?xt=urn:unknown:abc123', expected: false,
        desc: 'unknown urn type' },
  { input: 'magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36EXTRA',
        expected: false, desc: 'extra chars after hash' },
```

```
  // Edge cases
  { input: 'magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&xt=urn:btih:aaaaaaaa
        expected: true, desc: 'multiple xt' },
  { input: 'magnet:?
        xt.1=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&xt.2=urn:sha1:YNCKH
        expected: true, desc: 'numbered xt params' },

  // Within text
  { input: 'Check out this magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&dn=file
        for download', expected: true, desc: 'embedded in text' },
];
```

---

4. URI Decoding & Parameter Handling

4.1 URL Encoding Rules

Claim: The `tr`, `xs`, `as`, and `ws` parameters contain encoded URIs and must be decoded. The `dn` parameter uses `+` for spaces (application/x-www-form-urlencoded convention). Source: webtorrent/magnet-uri source code URL: https://github.com/webtorrent/magnet-uri/blob/master/index.js Date: 2023-05-31 Excerpt:

```
// Clean up torrent name
if (key === 'dn') val = decodeURIComponent(val).replace(/\+/g, '
        ')
// Address tracker (tr), exact source (xs), and acceptable source
        (as) are encoded
// URIs, so decode them
if (key === 'tr' || key === 'xs' || key === 'as' || key === 'ws')
        {
  val = decodeURIComponent(val)
}
```

Context: The official magnet-uri library's handling of URL-encoded parameters Confidence: high

4.2 Decoding Implementation

```
/**
 * Decode individual magnet URI parameter values
 * @param {string} key - Parameter name
 * @param {string} value - Raw (still URL-encoded) parameter value
 * @returns {string|number|string[]} - Decoded value
 */
function decodeMagnetParam(key, value) {
  switch (key) {
    case 'dn':
```

```
      // Display name: decode URI encoding, replace + with spaces
      return decodeURIComponent(value).replace(/\+/g, ' ');

    case 'tr':
    case 'xs':
    case 'as':
    case 'ws':
      // URL-encoded URIs
      return decodeURIComponent(value);

    case 'kt':
      // Keywords: decode then split on +
      return decodeURIComponent(value).split('+');

    case 'ix':
      // File index: cast to number
      return Number(value);

    case 'xl':
      // Exact length: cast to number
      return Number(value);

    case 'so':
      // Select only (BEP 53): decode then parse ranges
      return
        parseBep53Ranges(decodeURIComponent(value).split(','));

    default:
      // Unknown parameters: return as-is
      return value;
  }
}
```

4.3 Handling Duplicate Parameters

Claim: When the same parameter appears multiple times (e.g., multiple `tr` entries), they should be collected into an array. Source: webtorrent/magnet-uri source code URL: https://github.com/webtorrent/magnet-uri/blob/master/index.js Date: 2023-05-31 Excerpt:

```
// If there are repeated parameters, return an array of values
if (result[key]) {
  if (!Array.isArray(result[key])) {
    result[key] = [result[key]]
  }
  result[key].push(val)
} else {
  result[key] = val
}
```

Context: Standard handling for duplicate parameters in magnet URIs Confidence: high

---

5. xt Parameter Variations

5.1 Supported xt URN Types

URN Prefix	Hash Type	Encoding	Length	Network
`urn:btih:`	BitTorrent Info Hash (v1)	Hex	40 chars	BitTorrent
`urn:btih:`	BitTorrent Info Hash (v1)	Base32	32 chars	BitTorrent (legacy/Vuze)
`urn:btmh:1220`	BitTorrent Info Hash (v2)	Hex (multihash)	64 chars after prefix	BitTorrent v2
`urn:sha1:`	SHA-1	Base32	32 chars	Gnutella/G2
`urn:tree:tiger:`	Tiger Tree Hash	Base32	39 chars	Direct Connect/G2
`urn:bitprint:`	SHA-1.TTH	Base32	32 + 1 + 39	Gnutella/G2
`urn:ed2k:`	ED2K	Hex	32 chars	eDonkey2000
`urn:aich:`	AICH	Base32	32 chars	eDonkey2000
`urn:kzhash:`	Kazaa	Hex	32 chars	FastTrack
`urn:md5:`	MD5	Hex	32 chars	G2
`urn:crc32:`	CRC-32	Base10	10 chars	(rarely used)

5.2 Multiple xt Parameters

Claim: Multiple `xt` parameters can exist in a single magnet link using either repeated `xt=` or numbered `xt.1=`, `xt.2=` syntax. Source: Wikipedia - Magnet URI scheme URL: https://en.wikipedia.org/wiki/Magnet_URI_scheme Date: 2005-01-24 Excerpt:

```
xt also allows for a group setting. Multiple files can be included
by adding a count number preceded by a dot (".") to each link
```

```
parameter.
 magnet:?xt.1=[ URN of the first file]&xt.2=[ URN of the second
file]
```

Context: Standard allows both numbered and repeated parameter styles Confidence: high

5.3 Hybrid Torrent xt Handling

Claim: Hybrid torrents contain both v1 (`urn:btih:`) and v2 (`urn:btmh:`) xt parameters in the same magnet link. Source: libtorrent blog - BitTorrent v2 URL: https://blog.libtorrent.org/2020/09/bittorrent-v2/ Date: 2020-09-07 Excerpt:

```
It is possible to include both a v1 (btih) and v2 (btmh) info-hash
in a magnet link, for backwards compatibility.
```

Context: Hybrid torrents are backwards compatible, participating in both v1 and v2 swarms Confidence: high

---

6. dn Parameter Extraction

6.1 Display Name Decoding

```
/**
 * Extract and decode the display name from parsed magnet
       parameters
 * @param {Object} parsed - Parsed magnet URI object
 * @returns {string|null} - Decoded display name or null
 */
function extractDisplayName(parsed) {
  if (!parsed.dn) return null;

  // dn is already decoded by the parser
  // decodeURIComponent handles %XX sequences
  // + is converted to space (application/x-www-form-urlencoded
       convention)
  return parsed.dn;
}

/**
 * Validate that a display name is safe to render
 * @param {string} name - Raw display name
 * @returns {string} - Sanitized display name
 */
function sanitizeDisplayName(name) {
  if (!name || typeof name !== 'string') return '';

  // Remove control characters and potentially dangerous sequences
```

```
  // Keep: alphanumeric, spaces, common punctuation, Unicode
  // Remove: null bytes, control chars, HTML tags
  return name
    .replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/g, '') // Control
        chars
    .replace(/[<>]/g, '') // HTML tags
    .trim()
    .substring(0, 255); // Limit length
}
```

6.2 Unicode Handling

Claim: The display name should be UTF-8 encoded and URL-escaped for non-English characters. Source: BitComet documentation URL: https://wiki.bitcomet.com/inside-bitcomet/ Date: Unknown Excerpt:

```
dn* is the user-friendly display name (which may be displayed
while waiting for metadata); it should be UTF8 + URL_Escape
encoded for non-English characters.
```

Context: dn should be treated as UTF-8 with URL encoding for special characters Confidence: high

---

7. tr Parameter Handling

7.1 Tracker URL Decoding

```
/**
 * Extract and normalize tracker URLs from parsed magnet
 * @param {Object} parsed - Parsed magnet URI object
 * @returns {string[]} - Array of decoded tracker URLs
 */
function extractTrackers(parsed) {
  if (!parsed.tr) return [];

  const trackers = Array.isArray(parsed.tr) ? parsed.tr :
        [parsed.tr];

  return trackers
    .map(tr => {
      try {
        // Already decoded by the parser, but validate URL format
        return validateTrackerUrl(tr) ? tr : null;
      } catch {
        return null;
      }
    })
    .filter(Boolean); // Remove nulls
```

```
}

/**
 * Validate a tracker URL
 * @param {string} url - Tracker URL
 * @returns {boolean}
 */
function validateTrackerUrl(url) {
  if (!url || typeof url !== 'string') return false;

  try {
    const parsed = new URL(url);
    // Valid tracker protocols
    return ['http:', 'https:', 'udp:'].includes(parsed.protocol);
  } catch {
    return false;
  }
}
```

7.2 Tracker Protocol Support

Protocol	Scheme	Description
HTTP	`http://`	Traditional HTTP tracker announce
HTTPS	`https://`	Secure HTTP tracker (less common)
UDP	`udp://`	UDP tracker protocol (BEP 15) - more efficient

7.3 Deduplication

Claim: Tracker URLs should be deduplicated since the same tracker may appear multiple times. Source: webtorrent/magnet-uri source code URL: https://github.com/webtorrent/magnet-uri/blob/master/index.js Date: 2023-05-31 Excerpt:

```
// remove duplicates by converting to Set and back
result.announce = Array.from(new Set(result.announce))
```

Context: The library deduplicates trackers using Set Confidence: high

---

8. JavaScript/TypeScript Parser Implementation

8.1 Complete Custom Parser

```typescript
/**
 * Magnet URI Parser - Complete Implementation
 * Based on BEP 9, BEP 53 with v1/v2/hybrid support
 */

// Types
interface ParsedMagnet {
  // Raw parameters
  xt?: string | string[];
  dn?: string;
  tr?: string | string[];
  xl?: number;
  xs?: string | string[];
  as?: string | string[];
  ws?: string | string[];
  kt?: string | string[];
  mt?: string;
  so?: number[];
  'x.pe'?: string | string[];
  [key: string]: unknown;

  // Convenience properties
  infoHash?: string;        // 40-char lowercase hex v1 hash
  infoHashV2?: string;      // 64-char lowercase hex v2 hash
        (after 1220 prefix)
  infoHashBuffer?: Uint8Array;
  infoHashV2Buffer?: Uint8Array;
  name?: string;
  keywords?: string | string[];
  announce: string[];        // Deduped tracker list
  urlList: string[];         // Combined as + ws
  peerAddresses: string[];   // x.pe entries
}

/**
 * Base32 alphabet for RFC 4648
 */
const BASE32_ALPHABET = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ234567';

/**
 * Decode a base32 string to Uint8Array
 * Uses RFC 4648 alphabet (A-Z, 2-7)
 */
function base32Decode(input: string): Uint8Array {
  const cleaned = input.toUpperCase().replace(/=+$/, '');
```

```
  const output: number[] = [];
  let bits = 0;
  let value = 0;

  for (const char of cleaned) {
    const idx = BASE32_ALPHABET.indexOf(char);
    if (idx === -1) throw new Error(`Invalid base32 character: $
        {char}`);

    value = (value << 5) | idx;
    bits += 5;

    if (bits >= 8) {
      output.push((value >> (bits - 8)) & 0xFF);
      bits -= 8;
    }
  }

  return new Uint8Array(output);
}

/**
 * Convert Uint8Array to hex string
 */
function bytesToHex(bytes: Uint8Array): string {
  return Array.from(bytes)
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}

/**
 * Validate hex string of specific length
 */
function isValidHex(str: string, length: number): boolean {
  return typeof str === 'string' &&
         str.length === length &&
         /^[a-f0-9]+$/i.test(str);
}

/**
 * Check if string is valid base32 (RFC 4648)
 */
function isValidBase32(str: string, length: number): boolean {
  return typeof str === 'string' &&
         str.length === length &&
         /^[A-Z2-7]+$/i.test(str);
}

/**
 * Detect and convert infohash from xt parameter
 * @returns Object with infoHash (hex) and infoHashV2 (hex), or
        null if invalid
```

```
 */
function extractInfoHashes(xt: string): { infoHash?: string;
        infoHashV2?: string } | null {
  // v1 hex: urn:btih: + 40 hex chars
  const v1HexMatch = xt.match(/^urn:bti[h]?:([a-fA-F0-9]{40})$/i);
  if (v1HexMatch) {
    return { infoHash: v1HexMatch[1].toLowerCase() };
  }

  // v1 base32: urn:btih: + 32 base32 chars
  const v1Base32Match = xt.match(/^urn:bti[h]?:([A-Z2-7]{32})$/i);
  if (v1Base32Match) {
    try {
      const bytes = base32Decode(v1Base32Match[1]);
      return { infoHash: bytesToHex(bytes) };
    } catch {
      return null;
    }
  }

  // v2 multihash: urn:btmh:1220 + 64 hex chars
  const v2Match = xt.match(/^urn:btmh:1220([a-fA-F0-9]{64})$/i);
  if (v2Match) {
    return { infoHashV2: v2Match[1].toLowerCase() };
  }

  return null;
}

/**
 * Parse BEP 53 select-only ranges
 * e.g., "0,2,4,6-8" => [0, 2, 4, 6, 7, 8]
 */
function parseBep53Ranges(parts: string[]): number[] {
  const result: number[] = [];

  for (const part of parts) {
    if (part.includes('-')) {
      const [start, end] = part.split('-').map(Number);
      for (let i = start; i <= end; i++) {
        result.push(i);
      }
    } else {
      result.push(Number(part));
    }
  }

  return result;
}

/**
 * Main magnet URI decode function
```

```
 */
export function decodeMagnetURI(uri: string): ParsedMagnet | null
        {
  // Support 'magnet:' and 'stream-magnet:' schemes
  const schemeMatch = uri.match(/^\s*(?:stream-)?magnet:\?(.+)/i);
  if (!schemeMatch) return null;

  const query = schemeMatch[1];
  const result: ParsedMagnet = {
    announce: [],
    urlList: [],
    peerAddresses: []
  };

  // Parse query parameters
  const params = query.split('&');

  for (const param of params) {
    const eqIdx = param.indexOf('=');
    if (eqIdx === -1) continue; // Skip params without =

    const key = param.substring(0, eqIdx);
    let val = param.substring(eqIdx + 1);

    // Decode based on parameter type
    if (key === 'dn') {
      val = decodeURIComponent(val).replace(/\+/g, ' ');
    } else if (['tr', 'xs', 'as', 'ws'].includes(key)) {
      val = decodeURIComponent(val);
    } else if (key === 'kt') {
      val = decodeURIComponent(val); // Will be split later
    } else if (['ix', 'xl'].includes(key)) {
      val = Number(val) as unknown as string;
    } else if (key === 'so') {
      val = parseBep53Ranges(decodeURIComponent(val).split(','))
        as unknown as string;
    }

    // Handle duplicate parameters
    if (key in result) {
      const existing = result[key];
      if (Array.isArray(existing)) {
        existing.push(val as string);
      } else {
        (result as Record<string, unknown>)[key] = [existing,
        val] as unknown[];
      }
    } else {
      (result as Record<string, unknown>)[key] = val;
    }
  }
```

```
  // Extract info hashes from xt parameter(s)
  const xts = result.xt ? (Array.isArray(result.xt) ? result.xt :
        [result.xt]) : [];

  for (const xt of xts) {
    const hashes = extractInfoHashes(xt);
    if (hashes) {
      if (hashes.infoHash) result.infoHash = hashes.infoHash;
      if (hashes.infoHashV2) result.infoHashV2 =
        hashes.infoHashV2;
    }
  }

  // Set convenience properties
  if (result.infoHash) {
    result.infoHashBuffer = new Uint8Array(
      result.infoHash.match(/.{2}/g)!.map(b => parseInt(b, 16))
    );
  }
  if (result.infoHashV2) {
    result.infoHashV2Buffer = new Uint8Array(
      result.infoHashV2.match(/.{2}/g)!.map(b => parseInt(b, 16))
    );
  }
  if (result.dn) result.name = result.dn;
  if (result.kt) result.keywords = result.kt;

  // Build tracker list (deduped)
  if (result.tr) {
    const trs = Array.isArray(result.tr) ? result.tr :
        [result.tr];
    result.announce = [...new Set(trs)];
  }

  // Build URL list (as + ws)
  const urlSources: string[] = [];
  if (result.as) urlSources.push(...(Array.isArray(result.as) ?
        result.as : [result.as]));
  if (result.ws) urlSources.push(...(Array.isArray(result.ws) ?
        result.ws : [result.ws]));
  result.urlList = [...new Set(urlSources)];

  // Build peer address list
  if (result['x.pe']) {
    const peers = Array.isArray(result['x.pe']) ?
        result['x.pe'] : [result['x.pe']];
    result.peerAddresses = [...new Set(peers)];
  }

  return result;
}

/**
```

```typescript
 * Encode object into magnet URI string
 */
export function encodeMagnetURI(obj: Partial<ParsedMagnet>):
        string {
  const params: string[] = [];

  // Build xt parameter(s)
  const xts: string[] = [];
  if (obj.infoHash) {
    xts.push(`urn:btih:${obj.infoHash.toLowerCase()}`);
  }
  if (obj.infoHashV2) {
    xts.push(`urn:btmh:1220${obj.infoHashV2.toLowerCase()}`);
  }
  if (obj.xt) {
    const xtArr = Array.isArray(obj.xt) ? obj.xt : [obj.xt];
    xts.push(...xtArr.filter(xt => !xt.startsWith('urn:btih:')
        && !xt.startsWith('urn:btmh:')));
  }

  // Deduplicate and add xt
  const uniqueXts = [...new Set(xts)];
  for (const xt of uniqueXts) {
    params.push(`xt=${xt}`);
  }

  // Add dn
  if (obj.name || obj.dn) {
    const dn = obj.name || obj.dn!;
    params.push(`dn=${encodeURIComponent(dn).replace(/%20/g, '+')}
        `);
  }

  // Add tr (trackers)
  const trackers = obj.announce || obj.tr;
  if (trackers) {
    const trArr = Array.isArray(trackers) ? trackers : [trackers];
    for (const tr of [...new Set(trArr)]) {
      params.push(`tr=${encodeURIComponent(tr)}`);
    }
  }

  // Add ws (web seeds)
  if (obj.urlList) {
    const wsArr = Array.isArray(obj.urlList) ? obj.urlList :
        [obj.urlList];
    for (const ws of [...new Set(wsArr)]) {
      params.push(`ws=${encodeURIComponent(ws)}`);
    }
  }

  // Add xl
```

```
  if (obj.xl) {
    params.push(`xl=${obj.xl}`);
  }

  return 'magnet:?' + params.join('&');
}
```

---

9. Validation Functions

9.1 InfoHash Validation

```
/**
 * Validate a v1 infohash (40 hex characters)
 */
export function isValidInfoHashV1(hash: string): boolean {
  return typeof hash === 'string' &&
         hash.length === 40 &&
         /^[a-f0-9]+$/i.test(hash);
}

/**
 * Validate a v2 infohash (64 hex characters, after 1220 prefix)
 */
export function isValidInfoHashV2(hash: string): boolean {
  return typeof hash === 'string' &&
         hash.length === 64 &&
         /^[a-f0-9]+$/i.test(hash);
}

/**
 * Validate base32-encoded v1 infohash (32 characters)
 */
export function isValidInfoHashBase32(hash: string): boolean {
  return typeof hash === 'string' &&
         hash.length === 32 &&
         /^[A-Z2-7]+$/i.test(hash);
}

/**
 * Validate a complete magnet URI string
 */
export function isValidMagnetURI(uri: string): boolean {
  if (!uri || typeof uri !== 'string') return false;

  // Must start with magnet:?
  if (!/^\s*magnet:\?/i.test(uri)) return false;

  // Must have xt parameter with valid btih or btmh
```

```
  const xtMatch = uri.match(/[?&]xt=(?:urn:bti[h]?:[a-fA-F0-9]
        {40}|urn:bti[h]?:[A-Z2-7]{32}|urn:btmh:1220[a-fA-F0-9]
        {64})(?:&|$)/i);
  if (!xtMatch) return false;

  return true;
}

/**
 * Strict validation that also parses and verifies all parameters
 */
export function validateMagnetStrict(uri: string): {
  valid: boolean;
  errors: string[];
  parsed: ParsedMagnet | null;
} {
  const errors: string[] = [];

  if (!uri || typeof uri !== 'string') {
    return { valid: false, errors: ['URI is empty or not a
        string'], parsed: null };
  }

  const parsed = decodeMagnetURI(uri);

  if (!parsed) {
    errors.push('Failed to parse magnet URI');
    return { valid: false, errors, parsed: null };
  }

  // Must have at least one infohash
  if (!parsed.infoHash && !parsed.infoHashV2) {
    errors.push('No valid infohash found (v1 or v2)');
  }

  // Validate v1 hash if present
  if (parsed.infoHash && !isValidInfoHashV1(parsed.infoHash)) {
    errors.push(`Invalid v1 infohash: ${parsed.infoHash}`);
  }

  // Validate v2 hash if present
  if (parsed.infoHashV2 && !isValidInfoHashV2(parsed.infoHashV2))
        {
    errors.push(`Invalid v2 infohash: ${parsed.infoHashV2}`);
  }

  // Validate trackers
  if (parsed.announce) {
    for (const tr of parsed.announce) {
      try {
        const url = new URL(tr);
        if (!['http:', 'https:', 'udp:'].includes(url.protocol)) {
```

```
          errors.push(`Unsupported tracker protocol: ${tr}`);
        }
      } catch {
        errors.push(`Invalid tracker URL: ${tr}`);
      }
    }
  }

  return {
    valid: errors.length === 0,
    errors,
    parsed
  };
}
```

---

10. Edge Cases

10.1 Edge Case Catalog

Edge Case	Description	Handling
Only xt, no dn	Magnet with just infohash	Parse successfully, name will be undefined
Only xt, no tr	Trackerless magnet (DHT-only)	Parse successfully, announce will be empty
Base32 hash	32-char base32 instead of 40-char hex	Convert to hex using base32 decode
Mixed-case hash	Upper/lowercase in hash	Normalize to lowercase
Duplicate trackers	Same tracker appears multiple times	Deduplicate using Set
Hybrid torrent	Both v1 and v2 xt present	Parse both infoHash and infoHashV2
Multiple xt	Same-type xt repeated	Process each, last one wins or combine
Numbered xt	xt.1, xt.2 style parameters	Parse as separate xt entries
Empty dn	`dn=` with no value	Set name to empty string
URL-encoded dn	dn with %20 or +	Decode properly
Unicode dn		

Edge Case	Description	Handling
	Non-ASCII display name	Use decodeURIComponent for UTF-8
stream-magnet:	Non-standard scheme	Accept if it follows magnet format
x.pe parameters	Bootstrap peer addresses	Collect into peerAddresses array
BEP 53 so	File selection ranges	Parse ranges like “0,2,4-6”
xs with btpk	BEP 46 mutable torrents	Extract publicKey from `urn:btpk:`
Truncated magnet	Link cut off mid-parameter	Parse what is available, may miss some params
Invalid hex chars	Hash contains non-hex characters	Reject or skip that xt parameter
Wrong hash length	Hash too short or too long	Validate against expected lengths
Missing magnet:?	Just “magnet:” without query	No parameters to parse, invalid
Parameters after hash	Extra text appended	Strip at first non-URL character

10.2 Edge Case Code Examples

```
// Edge case: magnet with only xt (minimal valid magnet)
const minimal = decodeMagnetURI('magnet:?
        xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36');
// Result: { infoHash: 'd2474e86...', name: undefined, announce:
        [], ... }

// Edge case: base32 hash
const base32 = decodeMagnetURI('magnet:?
        xt=urn:btih:QHQXPYWMACKDWKP47RRVIV7VOURXFE5Q');
// Result: { infoHash: 'bc96...' (converted hex), ... }

// Edge case: hybrid torrent
const hybrid = decodeMagnetURI(
  'magnet:?xt=urn:btih:631a31dd0a46257d5078c0dee4e66e26f73e42ac' +
  '&xt=urn:btmh:1220d8dd32ac93357c368556af3ac1d95c9d76bd0dff6fa9833ecdac3d53134efa
  '&dn=hybrid-test'
);
// Result: { infoHash: '631a31dd...', infoHashV2: 'd8dd32ac...',
        name: 'hybrid-test', ... }

// Edge case: trackerless (no tr parameters)
const dhtOnly = decodeMagnetURI(
```

```
  'magnet:?
       xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&dn=DHT+Only+Torrent'
);
// Result: { infoHash: 'd2474e86...', announce: [], name: 'DHT
       Only Torrent', ... }
```

---

11. Magnet Link Generation

11.1 From InfoHash + Name + Trackers

```
/**
 * Generate a magnet URI from component parts
 */
export function generateMagnetLink(params: {
  infoHash?: string;        // v1 hash (hex)
  infoHashV2?: string;      // v2 hash (hex, without 1220 prefix)
  name?: string;            // Display name
  trackers?: string[];      // Tracker URLs
  webSeeds?: string[];      // Web seed URLs
  size?: number;            // File size in bytes
}): string {
  const parts: string[] = [];

  // xt parameter(s)
  if (params.infoHash) {
    parts.push(`xt=urn:btih:${params.infoHash.toLowerCase()}`);
  }
  if (params.infoHashV2) {
    parts.push(`xt=urn:btmh:1220${params.infoHashV2.toLowerCase()}
       `);
  }

  if (parts.length === 0) {
    throw new Error('At least one infohash (v1 or v2) is
       required');
  }

  // dn parameter
  if (params.name) {
    parts.push(`dn=${encodeURIComponent(params.name).replace(/%20/
       g, '+')}`);
  }

  // tr parameters
  if (params.trackers) {
    for (const tr of params.trackers) {
      parts.push(`tr=${encodeURIComponent(tr)}`);
    }
  }
```

```
  // ws parameters
  if (params.webSeeds) {
    for (const ws of params.webSeeds) {
      parts.push(`ws=${encodeURIComponent(ws)}`);
    }
  }

  // xl parameter
  if (params.size !== undefined) {
    parts.push(`xl=${params.size}`);
  }

  return `magnet:?${parts.join('&')}`;
}
```

---

12. Hash Format Conversion

12.1 Base32 to Hex Conversion

Claim: Base32-encoded v1 infohashes (32 chars) must be decoded and re-encoded to hex (40 chars) for compatibility with most systems. Source: qBittorrent Wiki URL: https://github.com/qbittorrent/qBittorrent/wiki/How-to-convert-base32-to-base16-info-hashes Date: 2024-05-01 Excerpt:

```
For compatibility reasons, the same document states that clients
should also support the 32 character base32 encoded info-hash.
When adding such a magnet link, qBittorrent will automatically
convert it to a base16 hash.

Example:
magnet:?xt=urn:btih:WRN7ZT6NKMA6SSXYKAFRUGDDIFJUNKI2
Converted: b45bfccfcd5301e94af8500b1a1863415346a91a
```

Context: Shows the need for base32-to-hex conversion in practice Confidence: high

```
/**
 * Base32 alphabet (RFC 4648)
 */
const BASE32_CHARS = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ234567';

/**
 * Decode base32 string to Uint8Array (RFC 4648)
 */
export function base32ToBytes(base32: string): Uint8Array {
  const input = base32.toUpperCase().replace(/=+$/, '');
  const output: number[] = [];
  let bits = 0;
```

```
  let value = 0;

  for (const char of input) {
    const idx = BASE32_CHARS.indexOf(char);
    if (idx === -1) {
      throw new Error(`Invalid base32 character: "${char}"`);
    }
    value = (value << 5) | idx;
    bits += 5;
    if (bits >= 8) {
      output.push((value >>> (bits - 8)) & 0xFF);
      bits -= 8;
    }
  }

  return new Uint8Array(output);
}

/**
 * Convert Uint8Array to hex string
 */
export function bytesToHex(bytes: Uint8Array): string {
  return Array.from(bytes)
    .map(b => b.toString(16).padStart(2, '0'))
    .join('');
}

/**
 * Convert hex string to Uint8Array
 */
export function hexToBytes(hex: string): Uint8Array {
  if (hex.length % 2 !== 0) {
    throw new Error('Hex string must have even length');
  }
  const bytes = new Uint8Array(hex.length / 2);
  for (let i = 0; i < hex.length; i += 2) {
    bytes[i / 2] = parseInt(hex.substring(i, i + 2), 16);
  }
  return bytes;
}

/**
 * Convert base32 infohash to hex
 * Input: 32-char base32 string
 * Output: 40-char hex string
 */
export function base32ToHex(base32: string): string {
  const bytes = base32ToBytes(base32);
  if (bytes.length !== 20) {
    throw new Error(`Expected 20 bytes (SHA-1), got $
        {bytes.length}`);
  }
```

```
  return bytesToHex(bytes);
}

/**
 * Convert hex infohash to base32
 * Input: 40-char hex string
 * Output: 32-char base32 string
 */
export function hexToBase32(hex: string): string {
  const bytes = hexToBytes(hex);
  if (bytes.length !== 20) {
    throw new Error(`Expected 20 bytes (SHA-1), got $
        {bytes.length}`);
  }
  return bytesToBase32(bytes);
}

/**
 * Convert Uint8Array to base32 string (RFC 4648)
 */
export function bytesToBase32(bytes: Uint8Array): string {
  let bits = 0;
  let value = 0;
  let output = '';

  for (const byte of bytes) {
    value = (value << 8) | byte;
    bits += 8;
    while (bits >= 5) {
      output += BASE32_CHARS[(value >>> (bits - 5)) & 31];
      bits -= 5;
    }
  }

  if (bits > 0) {
    output += BASE32_CHARS[(value << (5 - bits)) & 31];
  }

  // Pad to multiple of 8 characters
  while (output.length % 8 !== 0) {
    output += '=';
  }

  return output;
}
```

12.2 Conversion Examples

```
// Base32 to Hex
base32ToHex('WRN7ZT6NKMA6SSXYKAFRUGDDIFJUNKI2');
// => 'b45bfccfcd5301e94af8500b1a1863415346a91a'
```

```
// Hex to Base32
hexToBase32('b45bfccfcd5301e94af8500b1a1863415346a91a');
// => 'WRN7ZT6NKMA6SSXYKAFRUGDDIFJUNKI2='

// qBittorrent example
base32ToHex('QHQXPYWMACKDWKP47RRVIV7VOURXFE5Q');
// => hex equivalent
```

---

13. Library Comparison

13.1 magnet-uri (WebTorrent)

Claim: `magnet-uri` is the most widely used npm package for magnet URI parsing, maintained by the WebTorrent project. Source: GitHub - webtorrent/magnet-uri URL: https://github.com/webtorrent/magnet-uri Date: 2026 (active) Excerpt:

```
Parse a magnet URI and return an object of keys/values.
npm install magnet-uri
```

Context: 235 stars, 49 forks, actively maintained Confidence: high

Pros: - Battle-tested, used by WebTorrent ecosystem - Handles v1 hex, v1 base32, and v2 multihash - Supports BEP 53 (select-only ranges) - Handles deduplication of trackers/web seeds - Supports BEP 46 mutable torrents (`urn:btpk:`) - ESM + CJS support

Cons: - Dependencies on `@thaunknown/thirty-two`, `bep53-range`, `uint8-util` - No built-in URL validation for trackers - Does not handle `stream-magnet:` scheme variants (only `magnet:`)

13.2 parse-torrent (WebTorrent)

Claim: `parse-torrent` is a broader library that parses torrent identifiers including magnet URIs, .torrent files, and info hashes. Source: GitHub - webtorrent/parse-torrent URL: https://github.com/webtorrent/parse-torrent Date: 2025 (active) Excerpt:

```
Parse a torrent identifier (magnet uri, .torrent file, info hash)
The return value of parseTorrent will contain as much info as
possible about the torrent. The only property that is guaranteed
to be present is infoHash.
```

Context: More comprehensive than magnet-uri, handles .torrent files too Confidence: high

Pros: - Handles magnet URIs, .torrent files, info hashes, HTTP URLs - Provides `toMagnetURI()` and `toTorrentFile()` for encoding - Remote torrent parsing (async) - More comprehensive output

Cons: - Larger bundle size (includes torrent file parsing) - More complex API - May be overkill if you only need magnet parsing

13.3 @ctrl/magnet-link (TypeScript Port)

Claim: `@ctrl/magnet-link` is a TypeScript port of `magnet-uri` with fewer dependencies. Source: GitHub - scttcper/magnet-link URL: https://github.com/scttcper/magnet-link Date: 2019-01-07 (TypeScript port) Excerpt:

```
Port of webtorrent/magnet-uri by feross that uses fewer
dependencies in typescript
npm install @ctrl/magnet-link
```

Context: Fewer dependencies, TypeScript-first Confidence: high

Pros: - TypeScript-first - Fewer dependencies than `magnet-uri` - Similar API

Cons: - Less widely used - May lag behind upstream updates

13.4 Comparison Summary

Feature	magnet-uri	parse-torrent	@ctrl/magnet-link	Custom
Parse magnet URIs	Yes	Yes	Yes	Yes
Parse .torrent files	No	Yes	No	No
v1 hex support	Yes	Yes	Yes	Yes
v1 base32 support	Yes	Yes	Yes	Yes
v2 multihash support	Yes	Yes	Yes	Yes
Hybrid torrents	Yes	Yes	Yes	Yes
BEP 53 (so)	Yes	Partial	Yes	Yes
BEP 46 (btpk)	Yes	No	Yes	No
Encode back to URI	Yes	Yes	Yes	Yes
TypeScript types	No	No	Yes	Yes
Bundle size	Small	Medium	Small	Smallest

Feature	magnet-uri	parse-torrent	@ctrl/magnet-link	Custom
Dependencies	3	More	Fewer	0

14. Performance Considerations

14.1 Scanning Large Pages

For browser extensions scanning web pages for magnet links:

```
/**
 * High-performance magnet link scanner for content scripts
 * Uses a two-phase approach: fast scan + detailed validation
 */
export class MagnetScanner {
  // Phase 1: Fast candidate detection - avoids expensive regex
  // This regex is intentionally simple for speed
  private static readonly CANDIDATE_RE = /magnet:\?\S+/gi;

  // Phase 2: xt parameter validation (applied only to candidates)
  private static readonly XT_RE = /xt=([^&]+)/;

  /**
   * Maximum text length to scan at once (prevents regex
       catastrophic backtracking)
   */
  private static readonly CHUNK_SIZE = 100000; // 100KB chunks

  /**
   * Scan text for magnet links with performance optimizations
   */
  scan(text: string): string[] {
    const results: string[] = [];
    const seen = new Set<string>(); // Deduplicate

    // Process in chunks to avoid regex issues on very large
       inputs
    for (let offset = 0; offset < text.length; offset +=
       MagnetScanner.CHUNK_SIZE) {
      const chunk = text.substring(offset, offset +
       MagnetScanner.CHUNK_SIZE + 200);
      // Add overlap to catch links that span chunk boundaries

      const candidates = chunk.match(MagnetScanner.CANDIDATE_RE);
      if (!candidates) continue;

      for (const candidate of candidates) {
        if (seen.has(candidate)) continue;
```

```
        const xtMatch = candidate.match(MagnetScanner.XT_RE);
        if (xtMatch && this.isValidXt(xtMatch[1])) {
          seen.add(candidate);
          results.push(candidate);
        }
      }
    }

    return results;
  }

  private isValidXt(xt: string): boolean {
    // Quick validation of xt parameter value
    return /^urn:bti[h]?:[a-fA-F0-9]{40}$/i.test(xt) ||
           /^urn:bti[h]?:[A-Z2-7]{32}$/i.test(xt) ||
           /^urn:btmh:1220[a-fA-F0-9]{64}$/i.test(xt);
  }

  /**
   * Scan DOM for magnet links in href attributes
   * Much faster than scanning innerText
   */
  scanDOM(root: HTMLElement = document.body): HTMLAnchorElement[]
        {
    // Use querySelectorAll for native-speed DOM traversal
    // Look for anchor elements with magnet: href
    const links = root.querySelectorAll('a[href^="magnet:"]');
    return Array.from(links) as HTMLAnchorElement[];
  }

  /**
   * Scan for magnet links in both DOM and text content
   */
  scanPage(): { fromDOM: HTMLAnchorElement[]; fromText:
        string[] } {
    return {
      fromDOM: this.scanDOM(),
      fromText: this.scan(document.body.innerText)
    };
  }
}
```

14.2 Performance Tips

1. **Use DOM-first approach**: `querySelectorAll('a[href^="magnet:"]')` is orders of magnitude faster than text scanning
2. **Text chunking**: For large pages, process text in chunks to avoid regex catastrophic backtracking

3. **Two-phase validation**: Use a fast regex for candidates, then validate with stricter rules
4. **Deduplication**: Always deduplicate results (same magnet may appear multiple times on page)
5. **Debounce scans**: In MutationObserver, debounce scans to avoid excessive CPU usage
6. **Worker thread**: For very large pages, consider scanning in a Web Worker

14.3 MutationObserver for Dynamic Content

```typescript
/**
 * Observe DOM changes to detect dynamically added magnet links
 */
export function observeMagnetLinks(
  callback: (links: HTMLAnchorElement[]) => void,
  root: HTMLElement = document.body
): MutationObserver {
  const observer = new MutationObserver((mutations) => {
    const newLinks: HTMLAnchorElement[] = [];

    for (const mutation of mutations) {
      for (const node of mutation.addedNodes) {
        if (node instanceof HTMLElement) {
          if (node instanceof HTMLAnchorElement &&
              node.href.startsWith('magnet:')) {
            newLinks.push(node);
          }
          const childLinks = node.querySelectorAll?.
         ('a[href^="magnet:"]');
          if (childLinks) {
            newLinks.push(...Array.from(childLinks) as
         HTMLAnchorElement[]);
          }
        }
      }
    }

    if (newLinks.length > 0) {
      callback(newLinks);
    }
  });

  observer.observe(root, {
    childList: true,
    subtree: true
  });

  return observer;
}
```

---

15. Security Considerations

15.1 XSS Prevention

Claim: When displaying magnet link data (especially the `dn` parameter), proper sanitization is required to prevent XSS attacks. Source: MDN Web Docs - XSS URL: https://developer.mozilla.org/en-US/docs/Web/Security/Attacks/XSS Date: 2025-12-15 Excerpt:

```
Like all XSS attacks, these two examples are possible because the
website:
1. Uses input that could have been crafted by an attacker
2. Includes the input in the page without sanitizing it.
```

Context: The dn parameter could contain malicious content if not properly sanitized Confidence: high

15.2 Security Best Practices

```
/**
 * Sanitize display name for safe HTML rendering
 */
function sanitizeForHTML(str: string): string {
  const div = document.createElement('div');
  div.textContent = str;
  return div.innerHTML;
}

/**
 * Validate and sanitize all parsed magnet data
 */
function sanitizeMagnetData(parsed: ParsedMagnet): ParsedMagnet {
  return {
    ...parsed,
    dn: parsed.dn ? sanitizeForHTML(parsed.dn) : undefined,
    name: parsed.name ? sanitizeForHTML(parsed.name) : undefined,
    // Only allow http:, https:, udp: protocols
    announce: parsed.announce?.filter(tr => {
      try {
        const url = new URL(tr);
        return ['http:', 'https:', 'udp:'].includes(url.protocol);
      } catch {
        return false;
      }
    })
  };
}
```

15.3 Content Security Policy

Browser extensions should use a strict CSP to prevent injected scripts:

```
{
  "content_security_policy": {
    "extension_pages": "script-src 'self'; object-src 'self'"
  }
}
```

Appendix A: Multihash Format Reference

Claim: The v2 infohash uses the multihash format with a `0x12 0x20` prefix, where `0x12` identifies SHA-256 and `0x20` (32) is the digest length in bytes. Source: Multiformats - multihash URL: https://github.com/multiformats/multihash Date: 2020 Excerpt:

> `The info-hash with the btmh prefix is the v2 info-hash in multi-hash format encoded in hexadecimal. In practice, this means it will have a two byte prefix of 0x12 0x20.`

Context: The multihash prefix allows future hash algorithm upgrades Confidence: high

Multihash Prefix Table

Code (hex)	Name	Length	Description
`0x11`	sha1	20 bytes	SHA-1 (v1 infohash)
`0x12`	sha2-256	32 bytes	SHA-256 (v2 infohash)
`0x13`	sha2-512	64 bytes	Not currently used

The full v2 infohash format: `urn:btmh:1220<64-hex-chars>` where `12` = SHA-256, `20` = 32 bytes.

Appendix B: BEP 53 (Select-Only Extension)

Claim: BEP 53 extends magnet links with a `so` parameter to specify which file indices to download. Source: BitTorrent.org BEP 53 URL: https://www.bittorrent.org/beps/bep_0053.html Date: 2017-05-24 Excerpt:

so=0,2,4,6-8 means select only, and the numbers are the file indices. Files are zero-indexed. Dashes mean inclusive ranges, so 6, 7, and 8 are also added.

Context: Allows “deep links” to specific files within a torrent
Confidence: high

Appendix C: Reference Magnet Links for Testing

```
// v1 hex - minimal
magnet:?xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36

// v1 hex - with all parameters
magnet:?
xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&dn=Leaves+of+G

// v1 base32
magnet:?xt=urn:btih:QHQXPYWMACKDWKP47RRVIV7VOURXFE5Q

// v2 multihash
magnet:?
xt=urn:btmh:122067EFB3C2FA20CC493979084FB44C6A93F79442AFA4A2E01C898
test

// Hybrid v1+v2
magnet:?
xt=urn:btih:631a31dd0a46257d5078c0dee4e66e26f73e42ac&xt=urn:btmh:12
test

// With web seed
magnet:?
xt=urn:btih:08ada5a7a6183aae1e09d831df6748d566095a10&dn=Sintel&ws=h

// Trackerless (DHT-only)
magnet:?xt=urn:btih:e940a7a57294e4c98f62514b32611e38181b6cae

// With BEP 53 select-only
magnet:?
xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&dn=multi-
file&so=0,2,4-6

// With peer addresses
magnet:?
xt=urn:btih:d2474e86c95b19b8bcfdb92bc12c9d44667cfa36&x.pe=192.168.1
```

Appendix D: npm Package Quick Reference

```
# Primary magnet parsing library
npm install magnet-uri

# Broader torrent parsing (magnet + .torrent files)
npm install parse-torrent

# TypeScript alternative with fewer dependencies
npm install @ctrl/magnet-link

# For base32 encoding/decoding
npm install @thaunknown/thirty-two

# uint8-util for hex/array conversions (used by magnet-uri)
npm install uint8-util
```

Document compiled from: BEP 9, BEP 53, webtorrent/magnet-uri source code, Wikipedia Magnet URI scheme, BitTorrent.org specifications, qBittorrent documentation, Stack Overflow discussions, libtorrent blog, and RFC 4648.